OpenAgenet / OAN Yellow Paper: Technical Architecture for Trust-Governed Resource Identity and Discovery

Jinliang Xu
China Academy of Information and Communications Technology, Beijing, China
xujinliang@caict.ac.cn; jlxufly@gmail.com

***Abstract*—This yellow paper describes the technical architecture of OpenAgenet / OAN. OAN is a protocol-neutral trust layer for open Agent interconnection and discoverable AI resource products. It specifies the role architecture, `did:oan` identity objects, registration workflow, governance-backed Root lifecycle enforcement, Root-verified package model, authorization-aware Discovery, Root-issued infrastructure authorization VCs, signed trusted invocation, Root's trust, data-distribution, and semantic-governance hub responsibilities, verification requirements, state transitions, security properties, implementation boundaries, and deployment considerations. The design is intended to support heterogeneous Agent frameworks and interaction protocols, including MCP, A2A, ANP-like systems, domain-specific Agent protocols, Skills, MCP Servers, and Tool/API resources. OAN does not define the entire business conversation among Agents or the native protocol of every resource; it defines how resource identities become admissible, discoverable, verifiable, and safe to approach before protocol-specific interaction begins.**

Public portal: https://www.openagenet.xyz/

Open-source project: https://github.com/OpenAgenet

***Index Terms*—OpenAgenet, OAN, did:oan, resource identity, DID, VC, discovery, signed envelope, trusted invocation**

1. Technical Scope

OAN specifies a trust-governed resource identity and Discovery architecture. The system is concerned with the admissibility of `did:oan` identity representations across registration, Root acceptance, package distribution, Discovery indexing, Discovery response verification, and pre-connection invocation or access. It deliberately does not define a complete task protocol, model orchestration framework, prompt strategy, tool execution API, ranking algorithm, artifact hosting model, or business ontology.

The technical scope can be expressed through the following invariant: a resource identity should become visible to a relying party only after it has been prepared by the resource provider, checked by a Registrar whose infrastructure status is active under governance and credentialed by Root, accepted by Root, distributed through a verifiable package, indexed by a Discovery node whose infrastructure status is also active and credentialed, and returned with evidence that can be checked before invocation or access. Technically, Root's role is decomposed into three hub responsibilities: trust anchoring, verified data publication, and semantic tag-tree governance. The third responsibility governs vocabulary, tag versions, and compatibility for registration and Discovery; it does not make Root a semantic query executor.

The core abstraction is **Trusted Resource**, not only Agent. A trusted resource is a product-shaped entity, represented by a `did:oan` DID Document and a Root-verified ResourcePackage, whose lifecycle and discovery evidence can be checked before use. The initial resource types are Agent Service, Skill, MCP Server, and Tool/API. This abstraction lets OAN implement a Resource First model: the protocol admits heterogeneous AI ecosystem assets through one trust path, while allowing each product type to keep its native runtime, artifact format, or interaction protocol.

The current formal route style is intentionally unversioned:

- Root: `/root/...`
- Registrar: `/registrar/...`, `/resources/...`, `/capability-tree`
- Discovery: `/discovery/...`, especially `/discovery/resources/...`
- CDN: `/cdn/resources/...`, `/cdn/documents/...`, `/cdn/metadata/...`

Route versioning can be introduced later through content negotiation, typed object versions, or explicit API prefixes. The current design keeps the route surface simple while placing version information in protocol objects where verification actually occurs.

This paper should be read as a technical architecture and protocol-semantics document. It uses normative language for verification requirements, authorization rules, lifecycle transitions, and failure behavior. It uses implementation-profile language for deployment topology, storage backend, queueing, indexing, public portal integration, and operational visibility. A conforming implementation may choose different internal technologies, but it should preserve the external semantics described here: verifiable resource identity, governed admission, Root-verified publication, scoped Discovery, and pre-connection verification.

2. OAN Protocol Stack

OAN should be read as a protocol stack rather than as one service API. The stack has several profiles, each of which can be implemented independently and combined by different operators. Its lower layers reuse the DID/VC family of identity and credential concepts [1], [2], [3], [4], while its upper layers

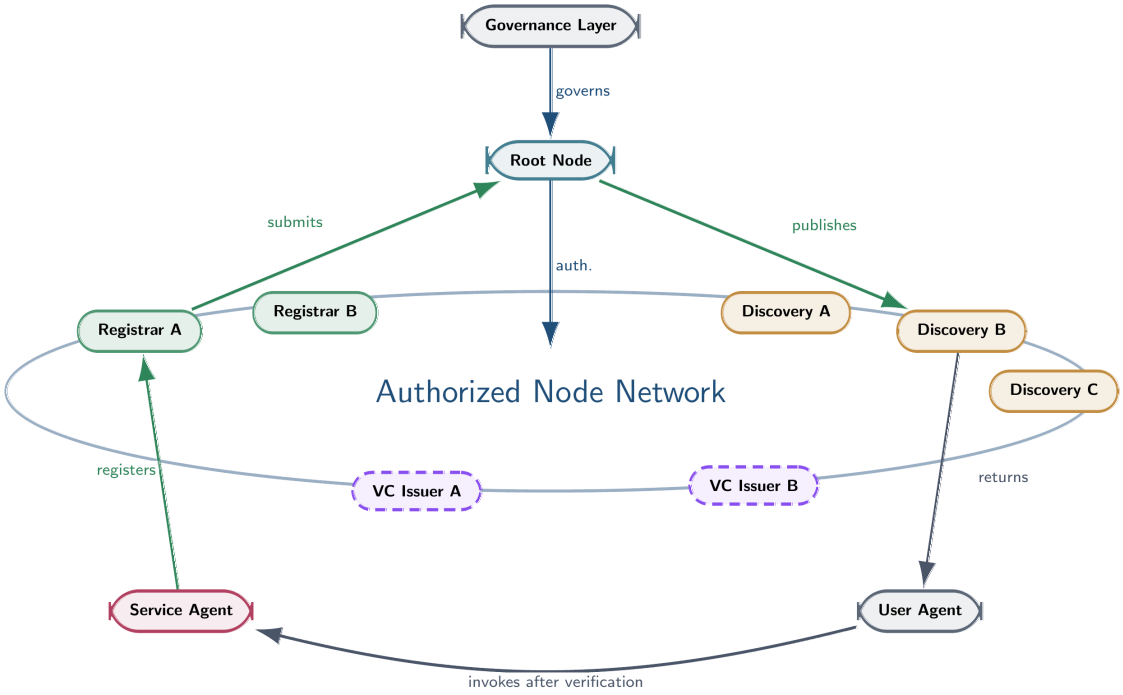


Fig. 1. OAN technical role framework. Governance authorizes infrastructure roles, Root enforces trust and publication authority, Registrars submit resources, Discovery nodes return verified candidates, and Agents invoke after verification.

TABLE I
OAN PROTOCOL STACK

Profile	Main objects	Primary users
Identity	`did:oan` DID, DID Document, OAN Identity, resource metadata	SDKs
Registration	draft, proof, credential	Registrar, Root
Federated governance	lifecycle state, threshold events	committee, Root
Root enforcement	Root VC, accepted record, package proof	Root, infrastructure
Package Discovery	verified package, cursor query, signed response	Root, CDN, Discovery Discovery, User Agent
Invocation/access	signed request/response, artifact reference	consumers/providers
Adapter	MCP/A2A/ANP mapping	protocol integrators

are designed to wrap Agent interaction protocols such as MCP and A2A [5], [6].

The stack structure clarifies implementation scope. A resource provider may only need the Identity, Discovery, and Invocation/access profiles. A Discovery operator needs Package and Discovery profiles plus Root bulletin verification. A full infrastructure operator needs Registration, Root Enforcement, Package, and Discovery. Governance operation clients additionally implement the Federated Governance profile. An adjacent protocol community can define an Adapter profile without changing the Root acceptance model.

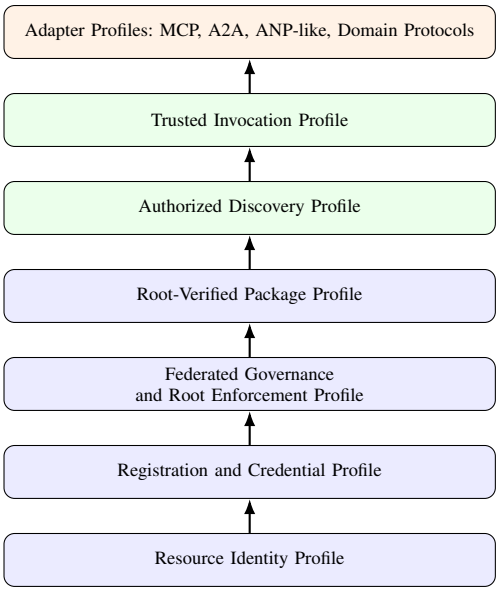


Fig. 2. OAN technical profiles form a layered trust stack.

3. SYSTEM ASSUMPTIONS

OAN assumes a trust domain with one Root service boundary, but not a single unreviewable Root decision maker. Infrastructure lifecycle state for Registrar, Discovery, and future VC issuer nodes can be governed above Root by a committee or threshold process and published through the on-chain

TABLE II
OAN TRUST LAYERS

Layer	Purpose	Mechanism
Federated governance	infrastructure lifecycle	threshold event
Root enforcement	protocol authorization	Root-issued VC
Registration review	resource admission workflow	Registrar credential
Subject control	prove DID ownership	challenge signature
Root acceptance	admit identity version	signed Root record
Package distribution	publish identity state	Root proof and hash
Semantic governance	govern tag tree	versioned taxonomy state
Discovery exposure	scoped query visibility	domain authorization
Runtime invocation	verify request peer	signed envelope

governance layer. Root consumes this state, enforces it, and issues protocol credentials. The current architecture therefore defines a federated-governance trust domain first. Federation among multiple independent trust domains or multiple Roots is treated as future work.

OAN also assumes that participants can perform standard cryptographic verification. Agents and infrastructure nodes are expected to hold DID-style identity material and signing keys. Signed objects use canonical serialization and hash binding. Time synchronization is assumed to be good enough for timestamp freshness windows, while nonce storage handles replay protection. The trust posture is aligned with zero-trust and workload-identity principles: every privileged role and runtime request must be explicitly verifiable rather than implicitly accepted by network location [7], [8].

The protocol-facing trust path should not depend on private review evidence such as email, phone, web account login, or manually reviewed forms. Such evidence may support governance workflow, but runtime verification should rely on DID documents, credentials, signed envelopes, Root bulletin facts, Root-verified packages, and Discovery response proofs.

4. TRUST LAYERS

OAN separates social governance, admission workflow, and runtime protocol verification. The layers are shown in Table II.

This separation makes failures easier to reason about. If a Registrar is misconfigured, Root still checks Registrar authorization and credential proof. If a CDN returns a tampered package, Discovery recomputes hashes and verifies Root proof. If a Discovery node is stale or unauthorized, User Agents can reject responses that fail freshness or authorization checks. If a caller replays an old invocation, the Service Agent can reject it using nonce and timestamp validation.

The threshold event in the first layer is a governance fact, not a protocol credential. A service node becomes protocol-authorized only after Root verifies the governance fact, validates the node's DID-control evidence, and issues the corresponding infrastructure authorization VC. The semantic-governance layer is also Root-mediated, but it is not a request execution path. It defines the governed semantic tag tree that Registrars use for registration-time guidance and that Discovery nodes use to maintain compatible semantic indexes.

5. CORE ENTITIES AND NOTATION

Let $\mathcal{A}$ be Agent subjects, $\mathcal{R}$ Registrar nodes, $\mathcal{D}$ Discovery nodes, $\mathcal{V}$ third-party VC issuer nodes, γ the governance layer, and ρ the Root service in one trust domain. For an infrastructure subject $s \in \mathcal{R} \cup \mathcal{D} \cup \mathcal{V}$, let $G_\gamma(s)$ be the latest governance status and $VC_\rho(s)$ be the Root-issued infrastructure authorization VC. Let $AD(x)$ be the typed authorized-domain set associated with subject or resource x. The subject is protocol-authorized only if:

$$InfraAuth(s) = Active(G_\gamma(s)) \wedge Valid(VC_\rho(s)).$$

This predicate is deliberately conjunctive. Governance approval without a Root VC is not yet a protocol credential, and a stale Root VC without active governance state is not sufficient for new infrastructure service. For Registrar and Discovery subjects, $AD(s)$ is also part of the governance and VC-bound authorization state. For VC issuer subjects it remains empty unless a later profile gives that role a domain-scoped authority.

A resource r has a versioned identity representation M_r^v. The representation hash is $H(M_r^v)$ and the resource domain set is $AD(M_r^v)$. A Root-accepted record contains at least:

$$\langle did_r, v, H(M_r^v), AD(M_r^v), status, t, \sigma_\rho \rangle.$$

The record binds resource DID, version, representation hash, authorized domains, lifecycle status, timestamp or logical time, and Root signature. The exact serialization may evolve, but the binding requirement is stable. Resource acceptance is Root anchored, while infrastructure eligibility is governance-backed and Root-credentialed.

An identity representation is admissible to a Discovery node d only if it is well formed, anchored by Root, current, and authorized for d:

$$\begin{aligned} Admissible(d, M) = {} & WellFormed(M) \wedge Anchored(M) \\ & \wedge Current(M) \wedge Scope(d, M). \end{aligned}$$

This predicate separates business search from trust admission. A query may match many metadata fields, but only admissible candidates should be exposed.

Table III defines the preferred terms used in this yellow paper. Product-facing labels such as Root Hub or Root Platform may appear in human-facing material, but the technical text should use Root Node or Root service when describing protocol behavior. Likewise, public portal and network transparency surfaces are interfaces over runtime state; they are not protocol entities.

6. ROLE ARCHITECTURE

The technical roles are intentionally separated.

TABLE III
STABLE TECHNICAL TERMS

Term	Meaning in this paper
Root Node	trust-domain service for acceptance, proof, publication, and infrastructure VCs
Registrar node	governed resource onboarding and Root submission node
Discovery node	package synchronization, indexing, query, and signed response node
Trust Indexer	component that converts governance events into runtime-readable authorization state
Public visibility surface	human-facing aggregation of metrics or summaries, not a protocol authority
Authorized domains	governance scope used for Registrar and Discovery authorization
Capability tags	semantic search and matching signals, not authorization scope

A. Root

Root owns accepted-version state, authorization-domain enforcement metadata, semantic tag tree governance, bulletin facts, infrastructure authorization VC issuance, and verified package generation. It consumes infrastructure lifecycle state from the on-chain governance layer and verifies Registrar authorization, signed upstream envelopes, registration credentials, DID structures, document hashes, capability metadata, and status transitions. Root should be the source of truth for current accepted resource identity versions, but not the sole governance source for infrastructure-node lifecycle legitimacy.

Root is not the business data plane. It should not need to observe every Agent-to-Agent call. This reduces operational load and avoids turning the trust anchor into a universal traffic proxy.

Root can be modeled as three coordinated hubs. As the trust authorization hub, it accepts resource versions, issues infrastructure credentials, and signs protocol evidence. As the data-distribution hub, it turns accepted identity state into Root-verified packages and publication cursors. As the semantic-governance hub, it governs the semantic tag tree used by Registrar for tag selection and recommendation and by Discovery for semantic matching, ranking, and explanation. The last role governs vocabulary and versions; it does not require Root to execute user-facing semantic queries.

In the upgraded design, Root acts as the policy enforcement point and protocol credential issuer for infrastructure nodes. A governance state transition does not by itself create a usable protocol credential. Root issues the infrastructure authorization VC after it observes active governance state and verifies DID-control and operational materials. Conversely, Root should stop serving or accepting later requests from an infrastructure subject whose latest governance state is inactive, even if the subject still holds an old VC.

Human-facing portal surfaces may present the same architecture as resource identity, Registrar admission, Root enforcement, package availability, Discovery indexing, and verification before invocation. In protocol terms, these surfaces are explanatory and operational views over DID documents, Root-verified packages, governance-derived authorization state, semantic tag-tree versions, Discovery response signatures, and signed invocation envelopes. They must not become alternative sources of trust.

B. Registrar

Registrar owns onboarding assistance, draft workflow, product-type declaration, semantic description collection, capability-tag suggestion, subject-control challenge handling, registration credential issuance, and Root submission. It is an operational gateway between resource providers and Root. Registrar can improve usability and review quality, but it does not unilaterally decide final Root acceptance.

C. Discovery

Discovery owns package synchronization, Root proof verification, bulletin checking, authorization-domain filtering, local indexing, query handling, and response signing. Discovery is allowed to optimize search and ranking, but its first obligation is to avoid exposing non-admissible identity representations. In the on-chain governance model, Discovery consumes authorization events through an indexer or local cache; it does not need to embed proposal or voting logic in the default service runtime.

D. CDN

CDN owns distribution of Root-verified packages and manifests. CDN improves availability and scalability but is not a trust authority. Consumers should verify package hashes, Root proof, metadata, and bulletin consistency.

E. Resource Provider and Resource Consumer

The Resource Provider controls a registered Agent Service, Skill, MCP Server, or Tool/API resource. The Resource Consumer is the relying Agent, application, developer tool, or human-facing client. Their trust relationship begins with Discovery verification and continues through signed invocation, credential checks, or artifact access according to the product type. OAN does not dictate the complete application conversation after the trust guard passes.

7. PROTOCOL OBJECTS

A. `did:oan` Identity Representation

The identity representation is centered on a `did:oan` DID Document compatible with DID Document concepts. A `did:oan` identifier has the form:

```
did:oan:<routing-code>:<suffix-code>
```

The `routing-code` is a method-specific routing or issuance-origin hint, while the `suffix-code` carries subject uniqueness. In the OAN reference trust path, infrastructure-node DIDs derive their routing relationship from the Root DID, and resource DIDs derive their routing relationship from the Registrar that admits the resource. This routing relationship is an admission and consistency check. It is not a proof of control, credential validity, or governance state by itself.

An OAN DID Document contains identifier, controller relationship, verification methods, authentication and assertion

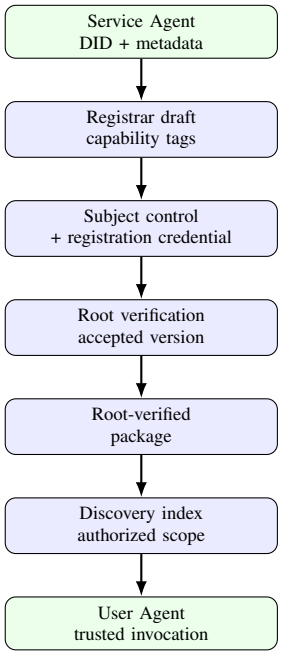


Fig. 3. OAN identity lifecycle from Agent preparation to trusted invocation.

relationships, service endpoints or artifact references, OAN metadata, and a Data Integrity proof. The default verification method is `<did>#key-1`. DID Document and VC proofs use the `Ed25519Signature2020` Data Integrity style with a multibase `proofValue`. The DID Document and VC proof fields do not use the older `creator`, `cryptoSuite`, or proof-level `hashAlgorithm` fields. Hash algorithm fields may still appear as business data when they describe resource-package or metadata digests; they are not proof-suite selectors.

The DID Document contains resource product type, service endpoints or artifact references, semantic description fields, capability tags, typed `authorizedDomains`, credential references, OAN metadata, and version metadata. The current OAN reference architecture standardizes on `did:oan` for resource identity and does not treat older method names as first-class protocol identifiers.

The identity representation should include enough material to support:

- key discovery and signature verification;
- endpoint or artifact-reference discovery for supported product types;
- authorized-domain validation and Discovery routing;
- semantic search over labels, descriptions, examples, and use cases;
- credential binding;
- version and status tracking;
- Root package generation and verification.

The core product types are Agent Service, Skill, MCP Server, and Tool/API. Agent Service, MCP Server, and Tool/API resources normally provide network endpoints. A Skill may instead provide an external artifact URI and enough semantic metadata for discovery. OAN Discovery indexes the DID Document and Root-verified package metadata; it does not need to host external artifacts.

The DID string should identify the resource subject, not encode mutable deployment facts. Endpoint URLs, artifact locations, schema references, descriptions, capability tags, authorization domains, status, and version metadata belong in the DID Document, product metadata, or Root-verified package. This separation keeps the identifier stable while allowing endpoints, keys, resource descriptions, and package references to evolve through normal versioning and Root acceptance.

B. OAN Identity and Controller Relationships

OAN distinguishes public identity documents from local identity material. An OAN Identity is local private material that binds a `did:oan` identifier, the default verification method, a DID Document, public key material, and the corresponding private key material. It is used to create DID Document proofs, request proofs, VC proofs, or application login challenge signatures. It is not published as a DID Document and must not be treated as a credential by itself.

This distinction supports three controller cases. In a self-controlled resource, the DID Document's `controller` is the resource DID itself. A dedicated controller can also be registered as a normal OAN resource, often using a controller-oriented subject and resource type. A third resource may be controlled by an external controller DID, in which case a controller authorization proof binds the resource DID, controller DID, Registrar DID, document hashes, verification method, purpose, validity window, and nonce. Registrar and Root both verify this proof when it is required. A self-controlled resource does not need an external controller authorization proof.

C. External Identifier Compatibility

OAN identifiers remain the primary identifiers inside the OAN trust path, but a DID Document may also declare external identifiers. An `externalIdentifiers` item contains an `id`, which is the native identifier in an external platform or identification system, and an optional `resolutionServiceEndpoint`, which gives the resolution context. The `id` is a metadata string, not the DID Document's top-level `id`, not a DID URL, and not a JSON-LD node identifier. Values such as repository paths, platform-local handles, industrial identifiers, or object identifiers can therefore be represented without rewriting them into `did:oan` syntax.

External identifiers improve interoperability with existing platforms and industry identifier systems, but they do not change DID control, Root acceptance, authorized-domain scope, or Discovery trust. They are controller-signed declarations inside the DID Document and should be verified or resolved according to their own external systems when stronger assurance is required.

D. Product-Type Metadata

Product-type metadata declares how a resource should be interpreted without forcing every resource into the Service

TABLE IV
INITIAL OAN RESOURCE TYPES

Type	Native object	Required OAN material	Native responsibility
Agent Service	callable Agent	endpoint, keys, claims	task protocol
Skill	skill artifact	artifact URI/hash, semantics	package format
MCP Server	MCP endpoint	endpoint, protocol hints	MCP behavior
Tool/API	API endpoint	endpoint/schema reference	API semantics

Agent shape. The minimal model should include product type, version, semantic labels, natural-language description, example use cases, authorized domains, endpoint descriptors or artifact references, and protocol hints. Product-native schemas remain outside the OAN core. For example, MCP tool listings, OpenAPI documents, or Skill package files can be referenced by URI and verified by hash when needed.

The product-type field is not an authorization shortcut. A resource marked as an MCP Server or Tool/API still needs normal Registrar onboarding, Root acceptance, package verification, and Discovery authorization checks before it is exposed. Its `authorizedDomains` must remain explicit and must not be inferred from product type, endpoint hostnames, or capability tags.

Product-type metadata should separate *trust fields* from *search fields*. Trust fields include DID, verification methods, resource type, version, status, endpoint or artifact-reference hash, credential references, and critical extension markers. Search fields include labels, descriptions, examples, natural-language use cases, local tags, and ranking hints. A verifier may use search fields to rank results, but it must not let search fields change authorization, current-version, or signature decisions.

Implementations should preserve product-native differences without giving them different trust semantics. An MCP Server can keep MCP descriptors, a Skill can keep its package format, and a Tool/API can keep an OpenAPI or domain schema. OAN only requires that the native material be referenced, hashed, typed, and bound to the same DID, Registrar, Root, Discovery, and lifecycle checks.

E. ResourcePackage as the Registry Object

The ResourcePackage is the registry object that makes OAN resource-first. It is not just an endpoint listing or a marketplace item. It binds:

- the `did:oan` resource identifier and DID Document hash;
- resource type, version, status, authorized domains, and semantic metadata;
- endpoint descriptors or external artifact references;
- Registrar credential evidence and subject-control binding;
- Root acceptance proof, publication cursor, and package hash;
- optional schema records, such as an external Agent-schema record, when a deployment chooses to use them.

This package can carry an MCP Server entry, a Skill identity, an Agent Service, or a Tool/API descriptor through the same lifecycle. Product-native metadata can remain in its native format, but it is placed inside or referenced by a Root-verified envelope. This is the main technical difference between OAN and a narrow registry that only stores tool endpoints or Skill artifacts.

External artifact references should be explicit. A ResourcePackage that points to a Skill file, OpenAPI document, MCP descriptor, or other artifact should include at least URI, media type, digest algorithm, digest value, expected version, and optional retrieval policy. OAN does not guarantee that the remote artifact host is trustworthy; it guarantees that the reference and digest were part of the Root-verified package. Consumers must still verify the digest after retrieval.

F. Capability Metadata

Capability metadata has a canonical component and an extensible component. The canonical component maps to the governed authorization-domain vocabulary and the governed semantic tag tree. The extensible component allows application-specific capability tags, search hints, labels, or descriptions. Canonical authorized domains, not custom tags, determine Registrar admission and Discovery exposure. Governed semantic tags and custom tags may refine registration suggestions, search, and ranking within an authorized set, but they must not grant exposure outside authorized domains.

G. Registration Credential

The resource registration credential binds:

- issuer Registrar DID;
- subject resource DID;
- credential type and claims;
- registration binding, including DID-control evidence;
- issuance time, validity, and proof.

Root verifies issuer authorization, credential proof, subject binding, credential status, product-type metadata, and validity before accepting a submitted representation.

Infrastructure authorization credentials, resource registration credentials, and third-party business credentials are distinct VC types. They can share the same outer VC and Data Integrity proof model, but their issuers, subjects, and status services differ. A VC's `credentialStatus` points to the issuer's status service. A status value embedded in the VC is a signing-time or query-time projection, not an automatically updating state. Governance indexers or other independent evidence sources may be used for higher-assurance checks, but they do not replace the issuer status interface as the general VC status model.

H. Subject-Control Challenge

Subject-control proof prevents a Registrar from issuing a credential for a DID that the applicant cannot control. The challenge should bind draft identifier, subject DID, Registrar DID, expiration time, and nonce. The Agent signs the challenge using an assertion-capable verification method from its DID document. Registrar verifies the signature and stores evidence or a verifiable summary.

I. Signed Upstream Envelope

Privileged infrastructure writes, especially Registrar-to-Root submissions, use a signed upstream request envelope. The envelope covers sender DID, target audience, method, path, body hash, timestamp, nonce, purpose, and protocol version. Root verifies the sender's current authorization and checks replay state before processing the body.

J. Root-Verified Package

The package consumed by CDN and Discovery includes the DID Document, metadata, hashes, Root proof, publication cursor, subject version, status, and capability information. Discovery does not trust the CDN path alone; it recomputes hashes and verifies Root proof and bulletin consistency.

K. Discovery Response

Discovery responses are signed by the Discovery node and include returned candidates, provenance metadata, timestamp, query context, and proof. A relying User Agent can verify that the response came from a currently authorized Discovery node and that selected candidates trace back to Root-verified packages.

L. Root Identity Directory

Some applications need to verify that a controller or resource identity can authenticate with an OAN Identity. For this purpose, Root may expose an identity directory that projects Root-accepted identities and their verified public keys. The directory is a public-key discovery aid for applications such as forum login. It does not replace DID resolution, VC verification, resource authorization, or the controller proof used during resource registration.

An identity directory item minimally contains the subject DID, status, verified public keys, an update time, and a revision indicator. A resource count may be reported as operational metadata, but it is not a login authorization rule by itself. Applications verify login by checking the directory entry and then verifying a fresh challenge signature produced by the user's OAN Identity. They should not trust a user-uploaded DID Document as the source of login public keys without Root-accepted evidence.

The directory is also not a cross-resource control list. A controller that appears in the directory may authenticate to an application, but resource registration still requires the Registrar and Root to validate the applicable resource DID, controller relationship, controller authorization proof, DID Document hashes, authorized-domain scope, and replay protections. This distinction keeps login public-key discovery separate from registration authorization.

8. On-Chain Governance Layer

The upgraded OAN governance profile can use an on-chain governance layer as the event source for infrastructure-node lifecycle state. Related SSI systems have also explored distributed trust anchors and lifecycle coordination [9], [10], [11]. The scope is intentionally narrow. It manages lifecycle state for Registrar, Discovery, and future third-party VC issuer nodes. It does not manage resource registration records, full identity documents, resource metadata updates, Root key rotation, or CDN service coordination.

This layer is what prevents the architecture from degenerating into a traditional single-operator architecture. A committee or other threshold-governed operator set can decide infrastructure lifecycle state, while Root enforces the result and signs the protocol artifacts that ordinary OAN services understand. The governance layer is therefore the publication plane for federated trust decisions, and Root is the enforcement and credentialization plane.

A. Lifecycle Scope

The governance layer records and emits events for:

- Registrar authorization, authorized-domain updates, suspension, recovery, and revocation;
- Discovery authorization, authorized-domain updates, suspension, recovery, and revocation;
- third-party VC issuer authorization and revocation.

The event stream replaces repeated status checks against Root for these infrastructure lifecycle facts. Registrar, Discovery, and VC issuer services consume indexed governance events, rebuild local authorization state, and enforce the resulting state in their own service logic. Root also consumes the same indexed state so that online VC issuance, Registrar submissions, Discovery notification, and future VC-issuer authorization can fail closed when the latest governance state is inactive or too stale.

Authorized-domain payloads are typed governance data, not opaque extension metadata. The all-domain value is represented by `["*"]`; an empty list means no domain authorization. A parent domain covers descendants by dot-prefix hierarchy. The current implementation uses a two-level domain taxonomy derived from existing Agent capability classification work, including OASF-inspired categories, but OAN treats the resulting identifiers as its own governed authorization-domain vocabulary.

B. Governance Operation Boundary

In a governed deployment, governance operations are performed by designated governance clients rather than by ordinary Registrar, Discovery, or VC issuer service nodes. These clients create proposals, submit votes, refresh proposal status, and manage participant lifecycle state.

Registrar, Discovery, and VC issuer service nodes are event consumers by default. Their runtime should include event subscription or polling and local authorization-cache maintenance, but it should not need proposal, vote, or member-management code. This separation keeps service nodes simpler and keeps governance private-key handling outside ordinary service runtimes.

C. DID and VC Boundary

The on-chain governance layer does not store, parse, validate, or issue DID Documents or Verifiable Credentials. It

TABLE V
ON-CHAIN GOVERNANCE BOUNDARY

In governance-layer scope	Outside governance-layer scope
Registrar lifecycle	Resource DID Document storage
Discovery lifecycle	VC issuance or validation
VC issuer lifecycle	Resource metadata parsing
Discovery domain updates	Resource metadata and package content
Lifecycle events	Root key and CDN coordination

TABLE VI
CORE OBJECT RESPONSIBILITIES

Object	Producer	Consumer	Purpose
Identity doc	Resource provider	Registrar, Root	subject keys
Reg credential	Registrar	Root	onboarding proof
Bulletin event	gov. client	infra nodes	lifecycle fact
Verified package	Root	CDN, Discovery	accepted version
Discovery response	Discovery	Resource consumer	query provenance
Invocation envelope	Resource consumer	callable provider	request binding
Signed response	callable provider	Resource consumer	response origin

also does not model the internal schema of `did:oan`, resource identifiers, credential claims, credential proofs, resource metadata, or capability descriptions.

Fields such as subject identifier, policy hash, metadata hash, effective time, expiration time, and status are only lifecycle references. DID and VC semantics remain off-chain OAN protocol responsibilities implemented by Root, Registrar, Discovery, VC issuer services, and SDK verification logic.

Effective infrastructure authorization requires both an active governance state and a valid Root-issued infrastructure authorization VC. If governance state is inactive, revoked, unknown, or too stale to trust, a previously issued VC is not sufficient for new service. If governance state is active but Root has not issued a VC, the node has lifecycle approval but is not yet authorized at the OAN protocol layer. This distinction is important for interoperability: service nodes can validate ordinary protocol credentials, while infrastructure operators and indexers can validate the fresher governance state.

D. Trusted Invocation Envelope

Before invoking a callable resource such as an Agent Service, the Resource Consumer sends caller DID material, credentials, nonce, timestamp, target DID, request body hash, discovery provenance, and request proof. The Resource Provider checks caller identity, credential subject, target binding, body hash, signature, nonce uniqueness, and timestamp freshness. Non-callable resources such as Skill artifacts can still use the same discovery and package evidence before the consumer follows an external artifact reference.

Table VI shows the design discipline of OAN: each object has a producer, a verifier, and a trust purpose. Objects should not be overloaded. For example, a Discovery response does not create Root acceptance; it only presents signed query provenance over already verified packages. A registration credential does not make an Agent discoverable; it only supports Root admission.

TABLE VII
RESOURCE IDENTITY LIFECYCLE STATES

State	Owner	Meaning
draft	Registrar	identity under preparation
control-verified	Registrar	DID control proof passed
credentialed	Registrar	registration credential issued
submitted	Registrar/Root	Root submission in progress
accepted	Root	version accepted by Root
published	Root/CDN	package ready for sync
synced	Discovery	package received and verified
structured-indexed	Discovery	structured index updated
semantic-indexed	Discovery	semantic index updated or marked recoverable
discoverable	Discovery	admissible for query within scope
superseded	Root	replaced by newer version
revoked	Root	no longer admissible

9. LIFECYCLE STATE MACHINE

OAN treats identity as stateful. A typical resource identity moves through the states shown in Table VII.

Create and update should use the same full-document path. Root should verify the complete new representation rather than applying ambiguous partial patches. This makes current-version verification easier and reduces risk that old fields remain accidentally authoritative. The table also separates acceptance from visibility. `accepted` does not mean immediately discoverable. A resource may be accepted by Root, published to the distribution layer, synchronized by Discovery, indexed structurally, indexed semantically, and then exposed as a query candidate. These steps may complete at different times, but they should leave observable state so that operators can distinguish lag from rejection.

The lifecycle is not a one-time publish action. Implementations should handle update, suspension, recovery, revocation, and republish as explicit state changes with verifiable causes. A newer accepted version should supersede older versions for normal Discovery, a suspended or revoked resource should not remain visible after synchronization, and a republished resource should re-enter Discovery through the same package-verification and authorization checks rather than bypassing them as a cache update.

10. LIFECYCLE FLOW

The standard OAN lifecycle is:

1) Resource provider prepares a `did:oan` identity representation with explicit `authorizedDomains`.
2) Registrar assists draft creation, authorized-domain selection, and capability-tag selection.
3) Resource provider proves control of its DID key.
4) Registrar verifies that the final resource domains are covered by its own authorized domains and issues a resource registration credential.
5) Registrar submits the complete representation to Root with a signed upstream envelope.
6) Root verifies Registrar authorization, envelope proof, credential proof, subject binding, DID structure, and capability metadata.

7) Root archives the accepted version and creates a Root-verified package.
8) CDN distributes the package.
9) Root targets Discovery notification by resource `authorizedDomains` and Discovery authorized-domain coverage.
10) Discovery synchronizes packages, verifies Root proof and bulletin facts, applies authorized-domain filtering, and indexes eligible Agents.
11) User Agent queries Discovery and verifies the signed response.
12) User Agent invokes Service Agent with a trusted invocation envelope.
13) Service Agent verifies the envelope and returns a signed response.

11. Root Verification Requirements

Root verification should be deterministic and auditable. At minimum, Root should verify:

- signed upstream envelope integrity;
- sender Registrar DID and authorization status;
- timestamp freshness and nonce uniqueness;
- request body hash binding;
- registration credential proof and issuer binding;
- subject DID binding between credential and document;
- DID document structure and verification methods;
- resource `authorizedDomains` against the Registrar's authorized domains;
- capability tags against the governed taxonomy as search metadata, not authorization evidence;
- semantic tag references against the current or compatible governed tag tree version;
- version and status transition validity.

Rejection should be explicit. Silent acceptance with partial checks is more dangerous than a visible rejection because downstream Discovery and relying consumers may assume Root acceptance means the whole trust path was verified.

12. Discovery Authorization

OAN uses authorized domains to control which Registrar may admit a resource and which Discovery node may expose it. Let $C(X, Y)$ mean that authorized-domain set X covers every domain in set Y, where `["*"]` covers all domains and a parent domain covers descendants by hierarchy. Let $Auth(rg)$ be the authorized domain set for Registrar rg, $Auth(d)$ the authorized domain set for Discovery node d, and $AD(M)$ the resource authorized-domain set in identity representation M.

$$RegistrarScope(rg, M) = \begin{cases} true, & C(Auth(rg), AD(M)), \\ false, & otherwise. \end{cases}$$

$$DiscoveryScope(d, M) = \begin{cases} true, & C(Auth(d), AD(M)), \\ false, & otherwise. \end{cases}$$

Custom tags are allowed, but they cannot expand the governance domain. They are used only after coarse authorization for local refinement, ranking, or search. A resource with empty `authorizedDomains` is not admissible for normal registration unless a local onboarding policy computes an explicit covered subset before Root submission.

This rule is intentionally strict because capability labels are often noisy, user-facing, multilingual, or generated by operators. A label such as `finance`, `health`, or `coding` may be useful for semantic matching, but it cannot prove that a Registrar is allowed to admit the resource or that a Discovery node is allowed to expose it. Authorization must come from the governed domain vocabulary and the corresponding node authorization state. The governed semantic tag tree complements this authorization vocabulary. It helps Registrar nodes recommend consistent tags during registration and helps Discovery nodes map user intent to comparable resource metadata during semantic discovery, but it is not an authorization grant.

A. Extension Safety Rules

Extensions are useful for local search quality and domain integration, but they must not weaken authorization. The following rules should hold for custom tags, plugin metadata, semantic embeddings, ranking features, and domain-specific attributes:

- an extension must not cause Registrar or Discovery to admit or expose an identity outside the applicable `authorizedDomains`;
- an extension must not override Root acceptance, current-version state, revocation state, or package verification results;
- ranking and semantic matching must run only after admissibility checks;
- unknown non-critical extension fields may be ignored, while unknown critical fields must cause rejection by verifiers that do not support them;
- plugin output should be auditable enough for an operator to explain why an identity was indexed, rejected, ranked, or returned.

B. Synchronization Rules

Discovery synchronization should be incremental. A publication cursor or watermark allows Discovery to fetch new packages without repeatedly scanning the entire Root archive. Discovery should verify each package before indexing and should track rejected packages with reasons. A package that fails Root proof, hash, bulletin, status, or authorization checks should not enter the query index.

C. Query Rules

Discovery query can use keywords, capability tags, endpoint metadata, local indexes, or ranking signals. However, query matching should never bypass the admissibility predicate. The proper order is: verify package, check current state, enforce

Discovery authorization, index admissible representation, then apply query matching and ranking.

Semantic discovery is an implementation profile above this trust filter. GRAIL studies deep-granularity hybrid resonance and SLM-enhanced indexing for real-time Agent discovery [12]; OAN can use this kind of semantic indexing inside Discovery nodes to map natural-language user intent to verified resource metadata. The invariant is that semantic matching ranks or filters only candidates that are already admissible under Root proof, current-version state, and Discovery authorization.

When a governed semantic tag tree is available, Discovery should bind its semantic index to an explicit tag-tree version or compatibility range. A Discovery node may use embeddings, local synonyms, examples, and ranking features, but those local features should be traceable back to governed tags when they affect cross-node resource comparability.

A semantic result should therefore be inspectable. Discovery may explain why a candidate matched, return highlighted metadata, or expose the underlying DID Document and package evidence for client verification. These inspection fields improve user trust and debugging, but they do not replace signed Discovery responses or Root package verification.

13. Trusted Invocation

Trusted invocation is the bridge from discovery to protocol-specific interaction. OAN does not define every message that Agents exchange after the first verified request. It defines the guard that lets a Service Agent decide whether a caller and request deserve entry into the business protocol.

A Service Agent should verify:

- caller DID document and verification method;
- caller credential proof and subject binding;
- Discovery response proof and freshness, when included;
- selected-service provenance against Root-verified package data;
- target DID equals the invoked Service Agent DID;
- timestamp is inside an allowed freshness window;
- nonce has not been used before;
- body hash matches the received body;
- request signature verifies over the canonical envelope.

If these checks pass, OAN hands control to the application protocol. That protocol may be A2A, MCP-adjacent tool invocation, a domain API, or a custom workflow.

14. Security Properties

OAN targets the following properties:

- **Provenance**: accepted identity representations trace to a Root-authorized acceptance event.
- **Integrity**: tampered identity representations fail hash or proof verification.
- **Freshness**: stale, revoked, or superseded versions are not admissible after synchronization.
- **Authorization soundness**: compliant Discovery nodes do not index packages outside their authorized domains.
- **Replay resistance**: signed upstream writes and invocation requests bind timestamp, nonce, target, path, and body hash.
- **Controller non-impersonation**: a resource controller field is not sufficient by itself; externally controlled registration requires a controller authorization proof over the exact registration context.
- **Application-authentication separation**: login-style challenge verification uses Root-accepted public-key evidence, but it does not authorize resource registration or replace VC and package verification.
- **Role accountability**: each privileged decision is attributable to an authorized infrastructure identity.

15. Threat Model

A. Malicious or Unauthorized Registrar

An unauthorized Registrar may attempt to submit an identity to Root. Root rejects it because the signed upstream envelope signer is not active in Root authorization state. A malicious but authorized Registrar may issue poor credentials; OAN can detect invalid signatures, wrong subjects, and policy violations, but semantic review quality remains a governance problem.

B. Tampered Package Distribution

An attacker controlling a CDN path may return modified documents or metadata. Discovery recomputes hashes and verifies Root proof. Tampering should cause package rejection.

C. Overbroad Discovery Exposure

A Discovery node may try to expose packages outside its authorized domains. Compliant Discovery software should filter such packages before indexing. Relying parties can also verify Discovery authorization state. Non-compliant Discovery behavior remains auditable if returned candidates include provenance that contradicts authorization policy.

D. Stale Lifecycle Cache

A Registrar, Discovery, or VC issuer node may miss bulletin events because of network failure or local indexer downtime. The service should expose sync lag, last processed event sequence, and cache health. Authorization-sensitive actions should fail closed when lifecycle state is unavailable or too stale.

E. Replay and Wrong-Target Invocation

An attacker may replay a previously signed request or redirect a signed request to another Service Agent. Timestamp windows, nonce stores, target DID binding, path binding, and body hash binding reduce these risks.

F. Controller Impersonation

An attacker may publish a resource DID Document whose `controller` field names a well-known controller DID. OAN does not treat that string as authority. For externally controlled resources, the Registrar and Root must verify a controller authorization proof signed by the controller over the resource DID, controller DID, Registrar DID, document hashes, purpose, validity window, and nonce. Without that

proof, the claimed controller relationship is only untrusted metadata.

G. Login-Key Substitution

An attacker may try to log in to an application by uploading a DID Document that binds a chosen DID to the attacker's own public key. Applications that use OAN Identity for login should therefore obtain accepted public-key evidence from Root's identity directory or an equivalent Root-accepted projection and then verify a fresh challenge signature. A user-uploaded DID Document alone is not a sufficient login trust anchor.

H. Semantic Misrepresentation

An Agent may claim a capability that it does not perform well. OAN can bind and govern the identity claim, but it cannot fully prove business quality or truth of all semantic claims. Reputation, testing, certification, and runtime policy must complement OAN.

16. Compatibility and Adjacent System Boundaries

OAN is not a replacement for MCP, A2A, ANP, or other Agent protocols [5], [6], [13].

MCP can use OAN to verify the identity and authorization context of tool servers or clients before tool access. An MCP client may check that a server endpoint belongs to a Root-accepted resource identity. An MCP server may use OAN credentials to decide whether a caller is allowed to access a tool.

A2A can use OAN to verify Agent identity and discovery provenance before task exchange. An A2A Agent Card or capability description can be mapped into the OAN DID-style identity representation, while A2A remains responsible for task state and message semantics.

ANP-like routes and Agent DID methods can be mapped to OAN identity documents and lifecycle checks. OAN's distinctive contribution is not only having an Agent identifier; it is the governed identity lifecycle, Discovery authorization, package verification, and pre-connection trust guard around that identifier.

A. Registry, Directory, and Schema Boundary

Several adjacent systems solve neighboring but different problems:

- MCP registries make MCP servers and server metadata discoverable, but they do not by themselves define DID/VC-based governance, Root acceptance, or infrastructure authorization [14].
- Skill marketplaces package and distribute Skills, while OAN gives a Skill a governed resource identity, lifecycle state, and verifiable discovery path.
- Agent schema frameworks provide vocabulary for Agentic records, capabilities, Skills, and relationships [15]. OAN can embed or reference such records as package metadata.
- ADS-style systems emphasize distributed Agent directory and discovery functions [16]. OAN emphasizes the trust path before and around discovery: Registrar onboarding, Root acceptance, package proof, governance-bounded Discovery, and signed verification.
- ANS-style systems emphasize names for Agents [17]. OAN can carry names as aliases, but its core identifier is the `did:oan` resource DID.

This boundary can be summarized as follows: external schema records solve description, ADS solves directory, ANS solves naming, MCP solves tool interaction, Harness-style runtimes solve execution, and OAN solves the trust envelope that allows these objects to be registered, distributed, discovered, and approached safely.

TABLE VIII
Protocol Integration Boundaries

Protocol	Keeps responsibility for	OAN provides
MCP	tools and resources	peer identity trust
A2A	task exchange	trusted discovery
ANP	Agent networking	lifecycle governance
DID/VC	primitives	Agent-specific policy
Domain API	business action	pre-connection guard

B. MCP Mapping

An MCP server can be represented as an OAN MCP Server resource or as a service endpoint inside a `did:oan` identity representation. MCP tool metadata can be mapped to OAN service metadata and capability tags. Before MCP initialization or tool execution, a client can query OAN Discovery, verify the Discovery response, verify the Root package, and then bind the MCP endpoint to the verified resource identity. MCP remains responsible for tool listing, tool invocation, resource access, and streaming behavior.

C. A2A Mapping

An A2A Agent Card can be mapped to an OAN identity representation by binding Agent identifier, service endpoints, capability declarations, and verification methods. OAN Discovery can serve as the pre-session discovery layer. After the candidate is verified, A2A can take over task negotiation, message flow, task state, and long-running collaboration. This avoids forcing A2A to solve Root governance and Discovery authorization internally.

D. ANP and DID Method Mapping

ANP-like route structures and Agent DID methods can be used as identifier and networking substrates. OAN can consume such identifiers if they can be resolved into verification methods and service metadata. The OAN-specific part is the registration path, Root acceptance, verified package, and authorized Discovery semantics around that identifier.

17. Implementation Profiles

OAN implementations do not need to implement every role at once. The following profiles define expected implementation depth.

TABLE IX
IMPLEMENTATION PROFILES

Profile	Required capabilities	Typical implementer
Agent	verify Discovery, sign calls	Agent runtime
Discovery	sync packages, sign query	directory operator
Registrar	draft, credential, submit	onboarding operator
Root	enforce, accept, sign package	trust-domain operator
Adapter	map OAN to MCP/A2A/ANP	protocol integrator
Full stack	all infrastructure roles	trial network

The profile model is important for adoption. A partner can begin with an Agent profile, later operate a Discovery node, and only eventually operate Registrar or Root infrastructure. This makes OAN deployable as a layered ecosystem rather than an all-or-nothing platform.

18. IMPLEMENTATION BOUNDARY

The reference implementation separates responsibilities along the same boundaries as the protocol model. Root implements acceptance, proof issuance, package publication, distribution coordination, and trust-domain services. Registrar implements governed onboarding, DID Document validation, registration review, and Root submission. Discovery implements package synchronization, structured indexing, derived semantic indexing, scoped query, and signed Discovery results. A governance indexer converts low-frequency governance events into runtime-readable node authorization state. Client libraries help applications generate OAN Identity material, create signed envelopes, verify DID Documents and credentials, and consume Discovery responses without reimplementing every cryptographic rule.

This split keeps the technical semantics understandable without making any particular repository layout, build script, database schema, or operator runbook part of the protocol. The paper therefore treats implementation components as reference realizations of Root, Registrar, Discovery, governance indexing, and client-side verification roles, not as independent normative authorities.

A. Normative Boundary

The technical design distinguishes normative protocol requirements from reference implementation choices. Normative requirements include signature coverage, hash binding, nonce and timestamp validation, Root proof verification, current-version checks, Discovery authorization filtering, failure reason reporting, and fail-closed behavior for security-critical verification. An implementation that omits these checks is not implementing the OAN trust model, even if it exposes similar HTTP routes.

Reference implementation choices include concrete database engines, queue implementations, local cache layouts, index structures, deployment automation, observability surfaces, and default retry policies. These choices may vary across operators as long as the externally visible verification semantics and conformance results remain consistent.

Human-facing portals, maps, dashboards, and documentation sites are non-normative presentation surfaces. The normative protocol semantics are DID Document validation, resource-type consistency, Root package verification, authorization-domain enforcement, Discovery response signing, lifecycle-state handling, and verification failure behavior.

19. PERSISTENCE AND OPERATIONAL STATE

Root, Registrar, Discovery, and CDN each maintain state with different criticality.

- Root state is authoritative for authorization, accepted versions, publication, and bulletin facts.
- Registrar state is authoritative for draft workflow, subject-control evidence, credential issuance, and submission attempts.
- Discovery state is authoritative for local synchronization cursor, package verification results, rejected package reasons, and query indexes.
- CDN state is authoritative for distributed package availability and manifest history, but not for trust decisions.

Implementation backends are non-normative. A reference implementation may use different local or production storage engines, but a conforming deployment must preserve the state needed to recover authoritative and derived trust decisions.

Operational state should be classified by authority and recoverability. Accepted Root records, resource versions, Root proofs, Registrar submissions, Registrar statistics, Discovery synchronization cursors, structured indexes, semantic index status, rejected or skipped package records, and public metrics snapshots should survive process restart and host migration. By contrast, in-memory request caches, UI rendering state, and short-lived health probes can be recomputed. A production profile should therefore define which state is authoritative, which state is a durable derived index, and which state is a temporary cache.

The protocol requirement is not a specific database product. It is that a conforming deployment preserves enough state to recover Root acceptance, Registrar submission history, Discovery visibility, semantic indexing progress, and public visibility summaries after restart.

20. PERFORMANCE ENVELOPE

The prototype evaluation reported in the figures covers single-node scales up to 2000 identity representations and logical multi-node scales up to 1200 total identity representations. The reported results show correct lifecycle execution, negative-case rejection, authorized Discovery completeness, and internal degraded-baseline effects. Separately, the public reference deployment has exercised a larger resource corpus and multi-node registration and Discovery paths. These operational observations support the feasibility of the architecture, but they are not a replacement for a controlled production benchmark.

The main technical pressures are large-result query processing, publication and synchronization tails, durable queue behavior, and local Discovery indexing costs. In particular,

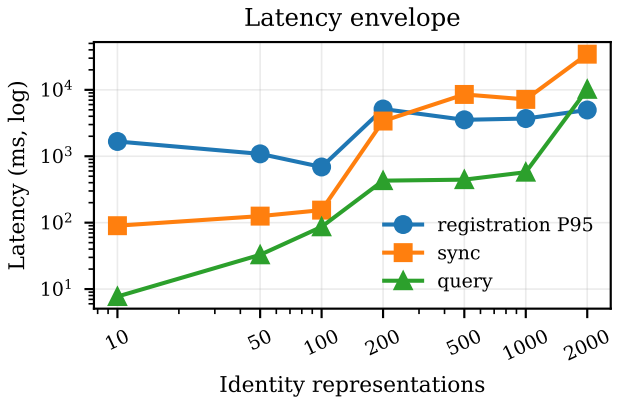


Fig. 4. Latency envelope across the single-node scalability experiment. The 2000-identity run exposes large-result query and synchronization pressure, which motivates pagination, index tuning, and queue hardening.

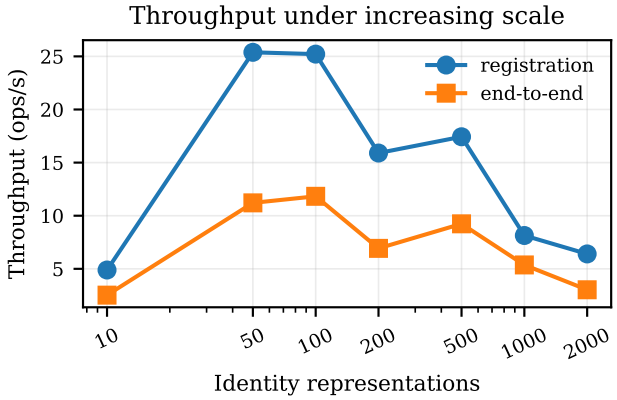


Fig. 5. Registration and end-to-end throughput under increasing identity scale. Throughput remains measurable through the tested range, while the gap between registration and end-to-end throughput highlights downstream publication, synchronization, and query costs.

Discovery may make a verified package visible through structured lookup before local semantic indexing has fully caught up. This decoupling is intentional: semantic ranking is a derived Discovery capability, not part of Root acceptance or package-verification trust.

The performance target of this yellow paper is not to claim final Internet-scale capacity. It is to define the technical architecture and expose where production hardening should focus: persistent queue tuning, query result pagination, index optimization, asynchronous derived-index recovery, distributed deployment, observability, database backend evolution, and more precise backpressure.

The ablation experiment was executed on copied reference-service workspaces so that the normal implementation remained unchanged. Each degraded variant removed one mechanism and then exercised the corresponding Root-node or Discovery-node code path over the same generated cases. The full path produced zero false acceptances or false exposures across the tested invalid cases, whereas the degraded variants

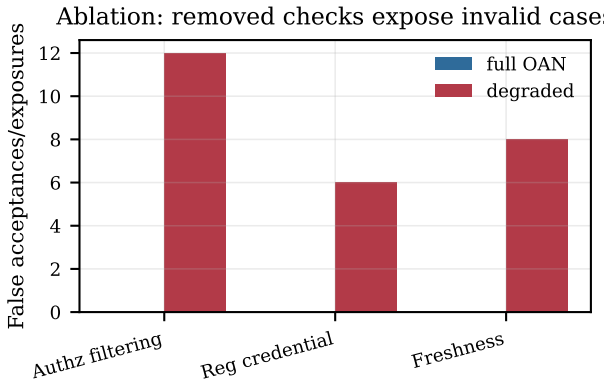


Fig. 6. Real-system degraded-baseline ablation. Removing authorization filtering, registration-credential verification, or freshness checks causes false exposure or false acceptance in the degraded variants, while the full OAN path rejects all tested invalid cases.

exposed all cases designed to target the removed check. These results support the claim that OAN's authorization, credential, and freshness checks are functional trust mechanisms rather than merely descriptive metadata.

21. Conformance Expectations

Independent implementations should be able to pass conformance tests for:

- `did:oan` identifier parsing, routing-code checks, DID Document profile validation, and canonical hashing;
- DID Document and VC proof verification using the current Data Integrity proof profile;
- OAN Identity generation and import checks that keep private key material local while binding it to the public DID Document;
- controller authorization proof verification for externally controlled resources and self-controller handling for self-controlled resources;
- external identifier parsing and validation without treating native external identifiers as OAN DIDs;
- product-type metadata for Agent Service, Skill, MCP Server, and Tool/API resources;
- external artifact reference digest verification;
- registration credential proof verification;
- subject-control challenge verification;
- signed upstream envelope verification;
- Root package verification;
- Discovery response signature verification;
- authorized-domain validation and authorization;
- nonce and timestamp replay rejection;
- current-version and revocation handling.

Conformance should include negative cases. A system that only succeeds on the happy path but accepts malformed credentials, stale packages, unauthorized Discovery exposure, or replayed requests is not OAN-compatible in the trust sense.

Conformance should also distinguish product-specific expectations from common trust expectations. A Skill confor-

mance vector may not require a network endpoint, but it must require artifact-reference integrity and semantic metadata. An MCP Server vector should require endpoint binding and protocol hints, but it should not require OAN to validate the full MCP tool list. A Tool/API vector should require endpoint or schema-reference binding, while API business semantics remain outside the OAN core.

22. API Semantics

This yellow paper does not freeze every HTTP route or JSON field, but it does define the semantic contract that APIs should preserve.

A. Root APIs

Root APIs can be divided into management, verification, publication, and read interfaces. Management interfaces update infrastructure authorization state, authorized-domain state, and operational configuration. Verification interfaces accept signed Registrar submissions and return deterministic accept or reject results. Publication interfaces create or expose Root-verified packages and publication cursors. Read interfaces expose status, bulletin facts, accepted versions, and authorized infrastructure state.

Root write APIs should require signed upstream envelopes. Root read APIs may be public or restricted depending on deployment, but the returned trust facts should be verifiable or derived from verifiable Root state. Root should not be treated as the public full-resource listing service. A Root read interface may expose verifiable Root state, package material, cursors, or accepted-version facts, but general resource enumeration and user-facing search belong to Discovery. Public network statistics belong to a visibility profile, not to Root protocol authority.

Root may also expose an identity-directory read interface that projects Root-accepted controller or resource identities and verified public keys for application authentication. This interface is a public-key discovery aid. It does not authorize resource control by itself and does not replace DID Document or VC verification.

B. Registrar APIs

Registrar APIs support draft creation, authorization-domain vocabulary retrieval, capability-tag suggestion, authorized-domain selection, subject-control challenge generation, controller-authorization challenge generation when an external controller is used, proof submission, credential issuance, and Root submission. A Registrar API should preserve the relationship between a draft, its DID document hash, its authorized domains, its subject-control proof, its credential, and its Root submission result.

Registrar should also expose enough status for a resource provider to understand why a draft cannot progress. For example, missing authorized domains, out-of-scope domains, missing capability tags, failed DID control, expired challenge, credential issuance failure, and Root rejection are different errors and should not be collapsed into one generic failure.

TABLE X
Representative Error Categories

Category	Example	Expected behavior
syntax	malformed object	reject before trust checks
crypto	bad signature	reject and log signer
authz	inactive Registrar	reject as unauthorized
freshness	stale timestamp	reject as expired
replay	repeated nonce	reject and audit
binding	wrong subject DID	reject as mismatch
scope	out-of-domain submission or package	reject or do not index
state	superseded version	hide from query
publication	package not synchronized	return pending or no result
semantic	semantic index pending or failed	fall back or mark recoverable
metrics	stale public cache	show stale status without changing trust
portal	public display unavailable	keep protocol services independent

C. Discovery APIs

Discovery APIs support status, synchronization, package inspection, rejected package inspection, and query. The resource-oriented route profile uses `/discovery/resources/...` and resource index/query routes rather than legacy Agent-only routes. Query responses should be signed and should include enough provenance for a User Agent to verify selected candidates. Discovery should not expose packages that have not passed package verification and authorization filtering.

If a Discovery node implements semantic matching, the API should make the semantic profile observable without making it part of the trust proof. For example, a response may include matched labels, embedding profile name, semantic score, or explanation fields inspired by SLM-enhanced indexing methods such as GRAIL [12]. These fields should be signed as response metadata for auditability, but verifiers must still base trust acceptance on Root package evidence, Discovery authorization, and candidate provenance.

Human-facing Discovery inspection surfaces are non-normative views over this API family. They may show resource DID, version, capability metadata, description, and DID Document material, but they do not replace SDK or client verification. Their protocol value is to keep Discovery output inspectable by a relying party before invocation.

D. CDN APIs

CDN APIs expose manifests, packages, documents, metadata, and publication history. CDN responses should be treated as distribution data, not trust decisions. Consumers should validate Root proof and hashes after fetching CDN content.

23. Error Taxonomy

Precise errors are part of interoperability. Table X summarizes common error categories.

Errors should be safe to expose. External responses may avoid leaking sensitive operator details, but internal logs should preserve enough information for audit and debugging.

A missing public search result should not be treated as one error class. It may result from Registrar submission rejection,

Root acceptance failure, publication lag, Discovery out-of-scope filtering, package verification failure, structured-index lag, semantic-index lag, query mismatch, or stale portal cache. Implementations should expose enough status to separate these cases without leaking private operational evidence.

24. Consistency Model

OAN does not require every component to become globally consistent instantaneously. The architecture uses a staged consistency model.

A. Root Consistency

Root is authoritative for accepted resource identity versions, archived representations, package generation, and Root-issued protocol credentials. Infrastructure authorization is a joint state derived from active governance status and valid Root-issued VC state. Within one Root deployment, state transitions should be transactional enough to avoid accepting a version without a corresponding archive and publication state. Root should also avoid ambiguous current-version state for the same resource DID and should fail closed when its governance-derived infrastructure cache is stale beyond the configured trust window.

B. Publication Consistency

Publication from Root to CDN and Discovery can be asynchronous. A newly accepted Agent may not be immediately queryable. This is acceptable if the system exposes publication cursors, synchronization status, and retry behavior. The important property is monotonic progress without silently losing accepted packages.

C. Discovery Consistency

Discovery is eventually consistent with Root publication. It should never compensate for staleness by accepting unverifiable packages. If it has not yet synchronized a new accepted version, it may return no result or an older current result depending on Root state and cursor timing, but it should not return a package that fails current-version checks once it has learned the superseding state.

D. Discovery Backfill and Semantic Index Recovery

A newly authorized Discovery node should be able to converge from an empty local index to the set of Root-accepted resources within its authorized scope. This backfill path should be idempotent: rerunning it must not duplicate resources, widen authorization scope, or overwrite newer verified state with older state. Backfill progress should be observable through Root accepted counts, Discovery synchronized counts, structured-indexed counts, semantic-indexed counts, pending or failed semantic-index records, and cursor lag.

Structured synchronization and semantic indexing are allowed to be decoupled. A semantic-index upsert failure should not roll back an already verified package synchronization, but it must leave durable retry state. Otherwise, a Discovery node may appear synchronized at the structured index level while remaining permanently incomplete for intent-based search. The required safety property is recoverable convergence: every accepted and in-scope package is either indexed semantically, marked pending, marked failed with reason, or excluded by an explicit authorization or lifecycle rule.

This decoupling is visible to operators but should not change the external trust semantics of Discovery. DID lookup, structured filtering, package provenance, and signed Discovery responses can be correct before semantic ranking has fully caught up. Semantic indexing is a derived search capability over already accepted structured state, not a precondition for Root acceptance or a replacement for package verification.

This section describes a consistency requirement, not a benchmark. It does not specify the embedding model, vector database, retry interval, or queue implementation. Those choices belong to implementation profiles as long as the observable state and recovery semantics are preserved.

25. Key Management Considerations

Every privileged role depends on signing keys. Key management is therefore a first-class operational issue.

A. Root Keys

Root signing keys anchor the trust domain. They should be protected with the strongest operational controls in the deployment. Rotation must be planned so that relying parties can distinguish old valid Root records from forged records. A production design should publish key identifiers, validity windows, and rotation records.

B. Registrar and Discovery Keys

Registrar keys sign upstream submissions and credentials. Discovery keys sign query responses. Both roles should support revocation and rotation. Root authorization state should refer to current infrastructure DIDs and verification methods, so that deactivated keys stop being trusted.

C. Resource and Controller Keys

Resource and controller keys prove subject control, authorize registration operations, and sign runtime requests or responses when the resource is callable. Local OAN Identity material may contain private keys, but public DID Documents, VCs, ResourcePackages, Discovery responses, and identity-directory entries must not expose private key material. Key rotation should be handled as an identity update. Root should accept the new identity version only after verifying the complete updated representation and supporting evidence.

26. Canonicalization and Hashing

OAN relies on hash binding. This makes canonicalization important. If two implementations serialize the same logical object differently, signatures and hashes may fail. Protocol objects should therefore define canonical JSON serialization or another deterministic representation. The following objects especially require stable canonicalization:

- DID-style identity representation;
- DID Document proof input and final DID Document hash;

- registration credential unsigned payload;
- signed upstream request envelope;
- Root-verified package metadata;
- Discovery response payload;
- trusted invocation envelope.

Test vectors should include canonical byte strings and expected hashes. This will reduce the risk that independent SDKs become incompatible despite using the same field names.

27. SDK Architecture

SDKs are necessary because OAN asks application developers to perform several security-sensitive checks. A good SDK should expose high-level functions while still allowing operators to inspect verification details.

A. Resource and Agent SDK

The Resource or Agent SDK should help resource providers and consumers create or import OAN Identity material, generate DID Documents, create signed envelopes, verify peer DID Documents, verify credentials, check timestamps and nonces, verify Discovery responses, and produce signed responses. It should not hide verification failures behind vague exceptions, and it must keep local private-key material separate from public DID Documents and credentials.

B. Discovery SDK

The Discovery SDK should help clients query Discovery nodes and verify signed responses. It should return both candidate data and verification status. A developer should be able to distinguish "no candidates matched" from "Discovery response failed verification".

Client configuration such as a `baseUrl` is a routing convenience, not a trust authority. An SDK may use it to choose a Registrar, Discovery node, or public portal endpoint, but it must still verify DID documents, Root package evidence, Discovery signatures, timestamps, versions, and authorization-derived status. Changing an endpoint should not silently change the trust decision unless the returned protocol evidence also changes.

C. Infrastructure SDK

The Infrastructure SDK should support Root, Registrar, Discovery, and CDN administration. It should build signed upstream requests, verify Root packages, interact with authorization-domain vocabularies, and support conformance tests. Governance operation support for the on-chain governance layer should be isolated in designated governance clients or processes. Default Registrar, Discovery, and VC issuer runtimes should depend on event subscription and authorization-cache APIs rather than proposal or voting APIs.

28. Deployment Topologies

A. Local Development

Local development can run one Root, one Registrar, one Discovery node, one CDN, one Service Agent, and one User Agent on a single machine. This topology supports reproduction of lifecycle tests, negative cases, and small benchmarks.

B. Single-Domain Pilot

A single-domain pilot may separate Root, Registrar, Discovery, and CDN into different services or machines while keeping one operator. This topology tests network behavior, synchronization, package distribution, and operational observability.

C. Reference Public Deployment Profile

The reference public deployment profile is a non-normative technical profile that demonstrates how the reference roles can be exposed in a publicly observable trust domain. A typical profile contains public interaction surfaces, Root, Registrar, Discovery, package distribution, governance-state indexing, persistent storage, and optional semantic-indexing components. The profile may be deployed in many physical forms and should not be interpreted as the only deployment shape.

The profile also demonstrates multiple Registrar and Discovery nodes in one trust domain. These nodes follow the same governance, authorization-domain, registration, distribution, synchronization, and Discovery rules. A node used for public pilot validation is not a distinct protocol role. It is an instance of the same Registrar or Discovery role with its own authorization and runtime state.

Deployment-specific hostnames, cloud-provider choices, internal ports, filesystem paths, account names, and secrets are outside the technical specification and should remain in operator-controlled runbooks.

D. Multi-Operator Trial

A multi-operator trial introduces independently operated Registrars and Discovery nodes. Root authorization state becomes more important, and authorized-domain boundaries should be tested carefully. This topology is the first meaningful step toward ecosystem deployment.

E. Production Trust Domain

A production trust domain should add durable databases, key management systems, rate limits, monitoring, backup, disaster recovery, release signing, and operator procedures. It should also define who can change Root authorization state and how emergency revocation is performed.

29. Observability

Observability should be designed around lifecycle and trust events rather than only HTTP status codes. Operators should track:

- Registrar draft state transitions;
- subject-control challenge success and failure;
- credential issuance counts and failures;
- Root acceptance and rejection reasons;
- Root publication queue depth and latency;
- CDN package availability and manifest history;
- Discovery synchronization cursor lag;
- Discovery package rejection reasons;
- query latency, result counts, and pagination behavior;
- nonce replay rejection and timestamp failures.

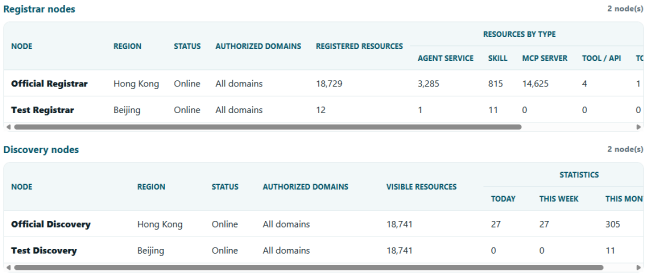


Fig. 7. Public node-level metrics as a non-normative visibility surface. The underlying protocol truth remains in Root acceptance, package verification, and Discovery authorization.

These metrics help distinguish performance bottlenecks from trust failures. For example, a missing Discovery result may be caused by query mismatch, publication lag, out-of-domain authorization, failed package verification, or Root rejection. Observability should make these causes separable.

A. Public Metrics Reporting

Public metrics reporting is a visibility profile, not a trust source. Registrar and Discovery nodes may publish or report public counters to a transparency surface. That surface may aggregate the latest node snapshots for human inspection, but clients must not treat those aggregates as authorization or acceptance facts.

Registrar metrics may include node identity, role, region, total registered resources, resources by type, and new registrations by time window. Discovery metrics may include node identity, role, region, visible resources, Discovery calls by time window, structured index state, semantic index state, and sync freshness. These fields support transparency, but they do not decide whether a resource is accepted, current, or callable.

B. Public Governance Event View

The public governance-event view is a curated transparency surface. It may show event time, affected node, and human-readable event description. It should not be used as runtime authorization input. Runtime authorization should be derived from indexed governance state, Root-issued infrastructure authorization material, and the verification logic of Registrar, Discovery, and relying clients. A human-readable event table is therefore explanatory, not normative.

30. Formal Invariants

The following invariants summarize the technical design.

A. I1: Root Acceptance Integrity

If Root accepts an identity version, then the accepted record must bind the resource DID, version, representation hash, status, and Root proof. Any later package claiming that acceptance must match this bound hash.

B. I2: Registrar Authorization

If Root accepts a Registrar-submitted identity, then the submitting Registrar must have been authorized at submission time and the signed upstream envelope must verify under the Registrar identity.

C. I3: Discovery Scope

If a compliant Discovery node indexes or exposes a resource identity, then the resource's authorized-domain set must intersect with the Discovery node's authorized-domain set or its descendants. Capability tags may refine matching and ranking, but they must not expand Discovery authorization.

D. I4: Current-Version Visibility

If a resource identity version has been superseded or revoked and Discovery has processed the relevant Root state, then that version must not remain visible as the current admissible version.

E. I5: Invocation Binding

If a Service Agent accepts a trusted invocation envelope, then the envelope signature, timestamp, nonce, target DID, body hash, and caller credential binding must have passed verification.

F. I6: DID Proof and Controller Authorization

If a DID Document is accepted into the OAN trust path, then its DID Document proof must verify under the declared verification method. If the resource is externally controlled, the controller authorization proof must also bind the resource, controller, Registrar, signed document hashes, purpose, validity window, and nonce.

G. I7: Root Identity Directory Non-Authority

If an application uses Root's identity directory for login public-key discovery, then a directory hit is not sufficient by itself. The application must still verify a fresh challenge signature under the listed public key. The directory does not authorize resource control or replace DID Document and VC verification.

If the same DID is used as a controller for another resource, that registration still requires the controller authorization proof defined for resource registration. Login success and controller authorization are intentionally different protocol decisions.

H. I8: Semantic Recovery

If structured synchronization succeeds but semantic indexing fails, the failure must leave durable retry state. A compliant Discovery node must not silently drop the semantic-indexing work item after accepting the verified package into its structured state.

I. I9: Public Surface Non-Authority

If a public portal, public metrics cache, or curated governance-event table is stale or unavailable, protocol truth must still be derived from Root acceptance, indexed governance state, package verification, Discovery authorization, and signed response verification.

31. Backward and Forward Compatibility

OAN should evolve without breaking every implementation. Compatibility can be managed through typed objects, optional fields, protocol version fields, capability negotiation, and conformance profiles. Critical verification fields should be mandatory. Experimental metadata should be optional and must not change trust semantics unless a new profile explicitly says so.

A useful compatibility rule is: old verifiers may ignore unknown non-critical metadata, but they must reject objects that require unknown critical trust semantics. This prevents silent downgrade when new security-relevant fields are introduced.

This rule is especially important for schema and directory integration. An external schema record, ADS-style directory hint, marketplace descriptor, or runtime-specific extension can be carried as non-critical metadata when it only improves description, search, or routing. It must become a critical extension only when a verifier is expected to use it for trust decisions, such as authorization scope, endpoint authenticity, version validity, revocation, or credential binding. This keeps OAN extensible without letting ecosystem metadata accidentally redefine the trust model.

A. Wire-Format Stability

Future protocol specifications should freeze the wire-level details that are only described semantically in this paper. These include canonical JSON rules, required and optional fields, field naming, object version identifiers, signature preimage construction, hash algorithms, HTTP method and path versions, pagination tokens, timestamp formats, nonce scope, error-code registry, and critical-extension markers.

Until those details are frozen, independent implementations should treat the reference objects and test vectors as the compatibility source of truth. A breaking field change should create a new object version or conformance profile. A non-breaking extension should be optional, ignored by old verifiers, and prohibited from changing trust decisions unless it is explicitly marked as critical and supported by the verifier.

32. Reference Verification Algorithms

The following algorithms describe expected verifier behavior at a high level. They are not tied to one programming language.

A. Root Submission Verification

Root submission verification should proceed in a fail-closed order. First, parse the request and reject malformed objects. Second, verify the upstream envelope signature and body hash. Third, check timestamp freshness and nonce uniqueness. Fourth, resolve the Registrar identity and confirm active authorization. Fifth, verify the registration credential, including issuer, subject, validity, and proof. Sixth, verify DID document structure, verification methods, capability metadata, and subject binding. Seventh, check version transition rules. Only after these checks should Root create an accepted record.

B. Discovery Package Verification

Discovery package verification should parse the package, recompute hashes, verify Root proof, check bulletin evidence, check current-version state, and apply authorized-domain authorization. Only packages that pass all checks should enter the query index. Failed packages should be retained in a rejected-package store with reason codes so that operators can debug synchronization and policy issues.

C. Discovery Backfill Verification

Discovery backfill should proceed as a repeated verification loop rather than as a blind database copy. First, load the Discovery node's current authorization state and synchronization cursor. Second, fetch candidate package identifiers or cursor ranges from Root or the distribution layer. Third, verify each package with the same package-verification algorithm used for incremental synchronization. Fourth, apply authorized-domain filtering before writing structured index state. Fifth, enqueue semantic indexing for accepted structured records. Sixth, record semantic success, pending, or failure state durably. Seventh, advance the cursor only according to the implementation's chosen cursor safety rule. Repeating these steps should be idempotent.

Backfill must not bypass the normal trust path. It should not import resources that fail Root proof, current-version checks, lifecycle checks, or Discovery authorization. It should also not treat semantic-index completion as a condition for Root acceptance. Semantic indexing affects intent matching and ranking; Root acceptance and Discovery authorization remain the trust gates.

D. User Agent Candidate Verification

A User Agent should verify the Discovery response signature, timestamp, Discovery authorization state, selected candidate provenance, Root package hashes, and current-version status before constructing an invocation. The User Agent should not treat Discovery output as plain search output. It is signed trust evidence that still requires validation.

E. Service Agent Invocation Verification

A Service Agent should verify caller identity, caller credential, target DID, request path, body hash, timestamp, nonce, and request proof. It should also apply local authorization policy after OAN checks pass. OAN proves who the caller claims to be and whether the request is fresh and bound; local policy decides whether that caller may perform the requested business action.

33. Pagination, Batching, and Backpressure

The reference experiments show that large-result queries, publication, and synchronization are important bottleneck areas. The architecture should therefore treat pagination, batching, and backpressure as protocol-relevant engineering concerns.

A. Pagination

Discovery query responses should support pagination when result sets become large. Pagination metadata must be covered by the Discovery response signature so that a client can verify page boundaries and query context. A page token should not allow a client to bypass authorization checks or retrieve stale results outside the original query policy.

B. Publication Batching

Root-to-CDN and Root-to-Discovery publication can be batched. Batch records should include cursor ranges, package identifiers, status, retry count, and error state. Batching should improve throughput without hiding individual package verification failures.

C. Backpressure

Registrar submission paths should have explicit backpressure. A burst of draft submissions should not create unbounded Root requests or local contention. Backpressure can include bounded queues, single-flight protection per draft, timeouts, retry policies, and clear submission-state reporting.

34. Degraded Behavior

Production systems must define behavior during partial failure.

A. Root Unavailable

If Root is unavailable, Registrar may continue draft preparation and local DID control checks, but it cannot claim Root acceptance. Discovery should continue serving previously verified current data within freshness policy, but it should not invent new Root facts.

B. CDN Unavailable

If CDN is unavailable, Discovery synchronization may lag. Existing verified index data may remain usable within policy, but new packages will not become visible until synchronization resumes. Operators should expose cursor lag and sync failures.

C. Discovery Stale

If Discovery is stale, User Agents may see incomplete results. This is safer than accepting unverifiable data. Discovery should publish status that allows clients and operators to understand synchronization lag.

D. Nonce Store Failure

If a Service Agent cannot check nonce uniqueness, it should fail closed for high-risk operations. For low-risk reference scenarios, a degraded mode may exist, but it should be explicit and should not be presented as full trusted invocation.

35. Data Retention

OAN deployments should define retention policies for drafts, credentials, accepted records, rejected submissions, package history, Discovery rejection logs, query logs, and invocation evidence. Retention should balance audit requirements with privacy and operational cost.

Root accepted records and revocation history usually require longer retention because they define trust-domain history. Registrar drafts and private review evidence may need shorter or policy-specific retention. Discovery query logs may contain sensitive interest patterns and should be minimized or protected. Invocation logs should be controlled by the Service Agent's business domain and privacy obligations.

36. Release and Supply-Chain Integrity

The OAN software supply chain should be treated as part of the trust model. Signed releases, reproducible build metadata, dependency review, artifact hashes, and compatibility statements help operators know which implementation they are running. Release practices should distinguish protocol-compatible changes from experimental or breaking changes.

Reference services should publish version information and protocol object support. SDKs should publish test-vector compatibility. Trial networks should record which service versions are active so that experiments and reference scenarios can be reproduced.

37. Operational Security Baseline

Production deployment requires more than passing reference integration tests. An OAN trust domain should define at least the following controls:

- durable nonce stores and timestamp skew policy for privileged writes and trusted invocation;
- event synchronization lag monitoring for Registrar, Discovery, and VC issuer authorization state;
- fail-closed authorization cache behavior when bulletin or Root state is unavailable or too stale;
- package verification failure alerts and rejected-package retention;
- Discovery index rebuild, cursor recovery, and query-result pagination;
- bounded queues, backpressure, retry policy, and timeout policy for Registrar submission and Root publication paths;
- key rotation, key compromise response, and operator access-control procedures;
- audit log retention for Root decisions, Registrar submissions, Discovery synchronization, and trusted invocation failures;
- release verification, dependency review, and conformance-test records for deployed service and SDK versions.

These controls do not change the protocol model, but they determine whether an OAN deployment can be trusted operationally by other organizations.

38. Test Vectors and Profiles

Interoperability requires more than prose. OAN implementations should publish test vectors that allow another implementation to verify whether it computes the same hashes, signatures, and authorization decisions.

A. Core Test Vectors

Core test vectors should include a canonical DID-style identity representation, its canonical byte sequence, its expected hash, a valid registration credential, an invalid credential with a wrong subject, a valid upstream envelope, an envelope with a mismatched body hash, a Root-verified package, a Discovery response, and a trusted invocation envelope. Each vector should include the expected verification result and the reason for rejection when applicable.

Additional vectors should cover resource and recovery semantics. These include a resource type mismatch between `did:oan` subject code and product metadata, an unauthorized Registrar submission, a Discovery node attempting to expose an out-of-domain resource, a revoked or superseded package, a Root proof mismatch, a semantic-index failure that leaves pending retry state, a repeated backfill operation that remains idempotent, and a stale public metrics cache that does not change protocol verification results.

B. Conformance Profiles

OAN can define conformance profiles for different implementation depths. A basic identity profile may only parse and verify identity documents and Root packages. A Discovery profile must additionally enforce authorized-domain authorization and sign responses. A trusted invocation profile must verify caller credentials, nonces, timestamps, target binding, body hashes, and request signatures. A full infrastructure profile must implement Root, Registrar, Discovery, and CDN semantics.

An operator profile should additionally preserve durable state, expose operational freshness, support Discovery backfill, and retain recoverable semantic-index failures. Public metrics may be included as a visibility extension, but they are not a prerequisite for the wire-format identity profile. These requirements test operational compatibility rather than wire-format identity alone.

C. Security Configuration Baseline

A production profile should define minimum security configuration. Examples include nonce retention windows, timestamp skew tolerance, maximum request body hash age, key rotation procedure, rejected-submission logging, Discovery sync lag alerts, package verification failure alerts, and disabled degraded modes for high-risk operations. These values may vary by deployment, but the presence of explicit configuration is itself part of operational maturity.

D. Interoperability Reports

Independent implementations should be able to publish interoperability reports. Such reports should state which profiles are supported, which test vectors passed, which optional extensions are implemented, and which known limitations remain. This creates a clearer ecosystem than informal claims of compatibility.

39. Privacy and Data Minimization

OAN identity representations are intended to be discoverable, but that does not mean every piece of operator evidence should be public. Private review evidence should remain in governance workflow or Registrar records unless there is a clear reason to publish it. Public packages should contain the material needed for verification and discovery, not unnecessary personal or operational data.

Future privacy work may include confidential queries, selective disclosure, private capability matching, scoped credentials, and policies for sensitive Agent categories. The current design leaves room for these extensions by separating Root acceptance, package publication, and Discovery exposure.

40. Open Technical Work

Important next steps include:

- multi-root and cross-trust-domain federation;
- richer semantic Discovery and authorization-domain taxonomy governance;
- privacy-preserving Discovery and confidential query processing;
- durable nonce stores and stronger replay windows;
- formal conformance tests for independent implementations;
- production deployment profiles and operational security baselines;
- MCP and A2A adapters with OAN pre-connection verification;
- stronger observability for Root, Registrar, Discovery, and CDN;
- package pagination and query-result pagination;
- signed release tooling and reproducible artifact verification.

41. Conclusion

OAN defines a technical architecture for governed resource identity and trusted Discovery. Its contribution is the coupling of federated infrastructure governance, Root-enforced resource provenance, current-version freshness, authorization-scoped Discovery, verified package distribution, and signed pre-connection validation. This coupling makes OAN closer to identity and governance infrastructure for the Agent Internet than to a plain Agent registry. It allows heterogeneous Agent protocols and implementations to interoperate on top of a shared trust substrate without forcing them into one task protocol or one Agent runtime. The Root service is the technical center of this substrate because it combines three bounded hub roles: trust authorization hub for accepted state and evidence, data-distribution hub for Root-verified packages

and cursors, and semantic-governance hub for the shared tag tree used across registration and Discovery.

The implementation lesson is that compatibility must be tested at trust boundaries, not only at HTTP endpoints. A compatible implementation should make the same decisions about DID document validity, resource-type binding, Registrar authorization, Root package proof, Discovery scope, lifecycle freshness, semantic-index recovery, and signed invocation. Human-facing surfaces, visibility dashboards, deployment automation, and storage choices can vary, but they should not redefine these decisions. This is the line that lets OAN remain open infrastructure instead of becoming one reference deployment.

Acknowledgment

The author thanks Jin Jian, Xie Jiagui, Li Bingqi, and Zhu Runkai for their support.

References

[1] W3C Credentials Community Group, "Decentralized identifiers (dids) v1.0," https://www.w3.org/TR/did-core/, 2022.

[2] W3C Verifiable Credentials Working Group, "Verifiable credentials data model v2.0," https://www.w3.org/TR/vc-data-model-2.0/, 2025.

[3] ——, "Verifiable credential data integrity 1.0," https://www.w3.org/TR/vc-data-integrity/, 2024.

[4] W3C Credentials Community Group, "Did resolution," https://w3c-ccg.github.io/did-resolution/, 2024.

[5] Model Context Protocol Contributors, "Model context protocol specification," https://modelcontextprotocol.io/specification/, 2025.

[6] A2A Protocol Contributors, "Agent2agent protocol specification," https://a2a-protocol.org/latest/specification/, 2025.

[7] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," NIST Special Publication 800-207, 2020.

[8] Cloud Native Computing Foundation, "The spiffe and spire project for workload identity," https://spiffe.io/, 2022.

[9] A. Grüner, A. Mühle, and C. Meinel, "Analyzing and comparing the security of self-sovereign identity management systems through threat modeling," in *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing*, 2023, pp. 389–396.

[10] M. Glöckler, J. Sedlmeir, V. Schlatt, and N. Urbach, "A systematic review of identity and access management requirements in enterprises and potential contributions of self-sovereign identity," *Business & Information Systems Engineering*, vol. 65, no. 4, pp. 421–440, 2023.

[11] M. Popa, X. Yue, A. Boudguiga, A. Chibani, and H. Bécha, "Chaindiscipline: A blockchain-based self-sovereign identity framework for iot," *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2571–2584, 2023.

[12] J. Xu, "Grail: A deep-granularity hybrid resonance framework for real-time agent discovery via slm-enhanced indexing," *arXiv preprint arXiv:2605.02489*, 2026.

[13] Agent Network Protocol Contributors, "Did:wba method specification: Web-based agent decentralized identifier," https://agentnetworkprotocol.com/en/specs/03-did-wba-method-specification/, 2025.

[14] Model Context Protocol Contributors, "Model Context Protocol Registry," https://github.com/modelcontextprotocol/registry, 2026, accessed: 2026-06-05.

[15] AGNTCY Contributors, "Open Agentic Schema Framework," https://github.com/agntcy/oasf, 2026, accessed: 2026-06-05.

[16] ——, "Agent Directory Service Overview," https://docs.agntcy.org/dir/overview/, 2026, accessed: 2026-06-05.

[17] K. Huang, V. S. Narajala, I. Habler, and A. Sheriff, "Agent name service (ans): A universal directory for secure ai agent discovery and interoperability," *arXiv preprint arXiv:2505.10609*, 2025.

[18] J. Xu and B. Li, "Darwinnet: An evolutionary network architecture for agent-driven protocol synthesis," *arXiv preprint arXiv:2604.01236*, 2026.

[19] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[20] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[21] D. B. Terry, M. M. Theimer, K. Petersen, A. J. Demers, M. J. Spreitzer, and C. H. Hauser, "Managing update conflicts in bayou, a weakly connected replicated storage system," *Proceedings of SOSP*, pp. 172–183, 1995.

[22] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and T. Polk, "Internet x.509 public key infrastructure certificate and certificate revocation list (crl) profile," RFC 5280, 2008.

[23] M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, "X.509 internet public key infrastructure online certificate status protocol - ocsp," RFC 2560, 1999.

[24] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "ToolEmu: Identifying the risks of lm agents with an lm-emulated sandbox," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=NB5FP1A2yC